

ARM PrimeCell

Synchronous Static Memory Controller (SSMC) PL093

Revision: r0p3-00rel0

User Guide



ARM PrimeCell Synchronous Static Memory Controller (SSMC) PL093

Revision: r0p3-00rel0

User Guide

© Copyright ARM Limited 2004. All rights reserved.

Proprietary notice

Words and logos marked with ® or ™ are registered trademarks or trademarks of ARM Limited in the EU and other countries, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product..

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- the document title
- the document number
- the page number(s) to which your comments refer
- a concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on this product

If you have any comments or suggestions about this product, please contact your supplier giving:

- the product name
- a concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	5
1.1	Change History	5
1.2	References	5
1.3	Terms and abbreviations	5
2	SCOPE	6
3	DIRECTORY STRUCTURE	6
3.1	Peripheral source code directory structure	7
3.2	Synopsys synthesis directory structure	8
3.3	BusTalk verification directory structure	9
3.4	Integration test directory structure	10
4	USAGE INSTRUCTIONS	11
4.1	Design tools used	11
4.2	Reference Cell library	11
4.3	Peripheral Development Environment	13
4.3.1	Unix environment variables	13
4.3.2	Makefile commands	14
4.4	Denali simulation models	15
4.4.1	Verilog-XL and Denali dynamically linked shared object libpli.so	15
4.5	Running simulations and synthesis	16
4.5.1	Generating BusTalk functional test vectors	17
4.5.2	Compiling peripheral RTL code	18
4.5.3	Running BusTalk verification RTL simulations	19
4.5.4	Running Synthesis	21
4.5.4.1	scr_common directory	22
4.5.4.2	scripts directory	22
4.5.4.3	log directory	22
4.5.4.4	report directory	23
4.5.5	Running netlist simulations	24
4.5.6	Generating the integration test vectors	25
4.5.7	Running the integration test RTL simulation	25
4.6	Error and warning messages	26
4.6.1	BusTalk RTL verification simulations	26
4.6.2	Bustalk netlist simulations	26
4.6.3	Integration test simulations	27

1 ABOUT THIS DOCUMENT

1.1 Change History

Issue	Date	Change
A		First release, supersedes PL000 USGD 0000 D generic User Guide
B	8 March 2003	r0p1 release
C	4 March 2004	Updated for r0p2 release
D	8 October 2004	Updated for r0p3 release

1.2 References

For further information refer to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0236	ARM PrimeCell SSMC PL093 Technical Reference Manual
3	PL093 DDES 0000	ARM PrimeCell SSMC PL093 Design Manual
4	PL093 INTM 0000	ARM PrimeCell SSMC PL093 Integration Manual
5	PL093 RELN 0000	ARM PrimeCell SSMC PL093 Release Note

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application Specific Integrated Circuit
IP	Intellectual Property
LDV	Cadence Tool Suite
RTL	Register Transfer Level
SSMC	Synchronous Static Memory Controller

2 SCOPE

This document describes the ARM Synchronous Static Memory Controller (SSMC) PL093 directory structure, peripheral development environment and the files within that structure. Instructions are given on how to run simulation and synthesis scripts and describes what output files are produced.

This document does not cover the installation of the PL093 deliverables. Please refer to the *ARM PrimeCell Synchronous Static Memory Controller PL093 Release Note* for installation instructions and a summary of the changes from the previous release.

For further information on the AMBA 2.0 protocol, refer to the *AMBA Specification (Rev2.0)*.

For details of the PL093 RTL implementation, testbenches and verification tests, please refer to the *ARM PrimeCell Synchronous Static Memory Controller PL093 Design Manual*.

3 DIRECTORY STRUCTURE

The peripheral development environment is stored in two directory structures, a peripheral specific directory structure and a project global directory structure. This emphasises that projects that contain multiple peripherals should be able to access the same shared data. The top level directory structures are shown in [Figure 3-1](#).

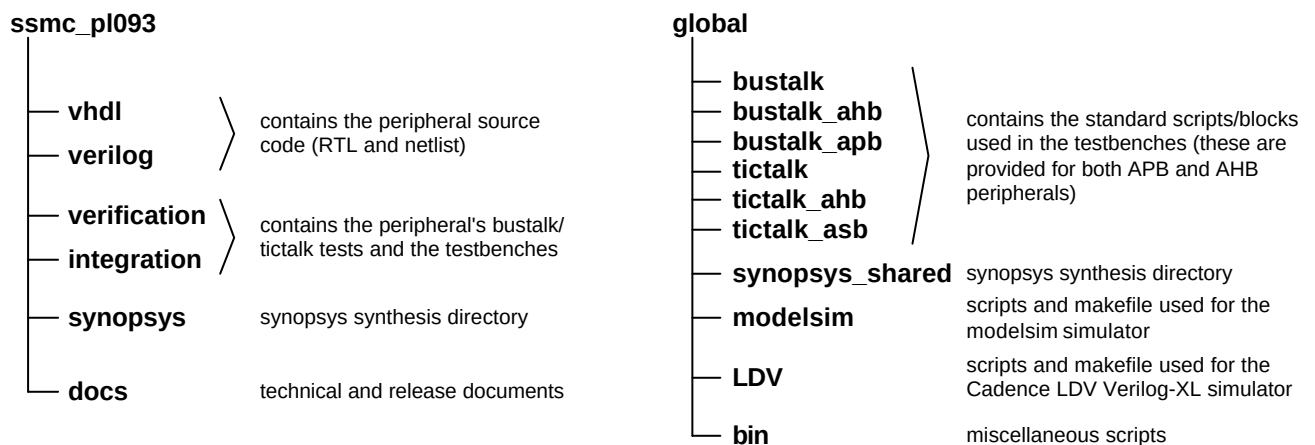


Figure 3-1: Peripheral Development Environment Directory Structure

3.1 Peripheral source code directory structure

The peripheral's source code is stored in two identical directory structures, as shown in [Figure 3-2](#).

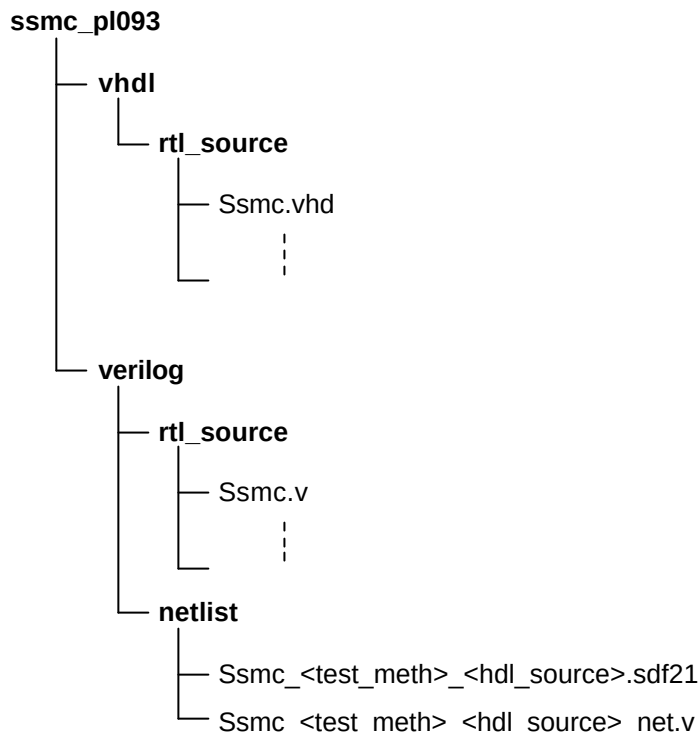


Figure 3-2: Peripheral source code directory structure

The source code is in the “rtl_source” directory and the synthesised netlists and associated SDF timing file are placed in the netlist directory. The `<test_meth>` and `<hdl_source>` parts of the netlist filename are derived from Unix environment variables. These are described in the section entitled [Unix environment variables](#) on page 13.

NOTE: The synthesis scripts supplied only write out Verilog netlists even when the RTL source is VHDL. Therefore, there is no netlist directory under `ssmc_pl093/vhdl` and simulations with a VHDL netlist are not directly supported in this release.

3.2 Synopsys synthesis directory structure

The synthesis command scripts and constraint files supplied support Synopsys Design Compiler and targets the Avant! Passport CB18 reference cell library. The synthesis environment directory structure is shown in [Figure 3-3](#).



Figure 3-3: Synopsys synthesis directory structure

NOTE The global/synopsys_shared/synopsys_cache directory does not exist in the delivered directory structure. This directory must be created manually as part of the synthesis process.

3.3 BusTalk verification directory structure

The verification directory contains the stand-alone BusTalk functional verification testbench and the BusTalk test program source as shown in [Figure 3-4](#).

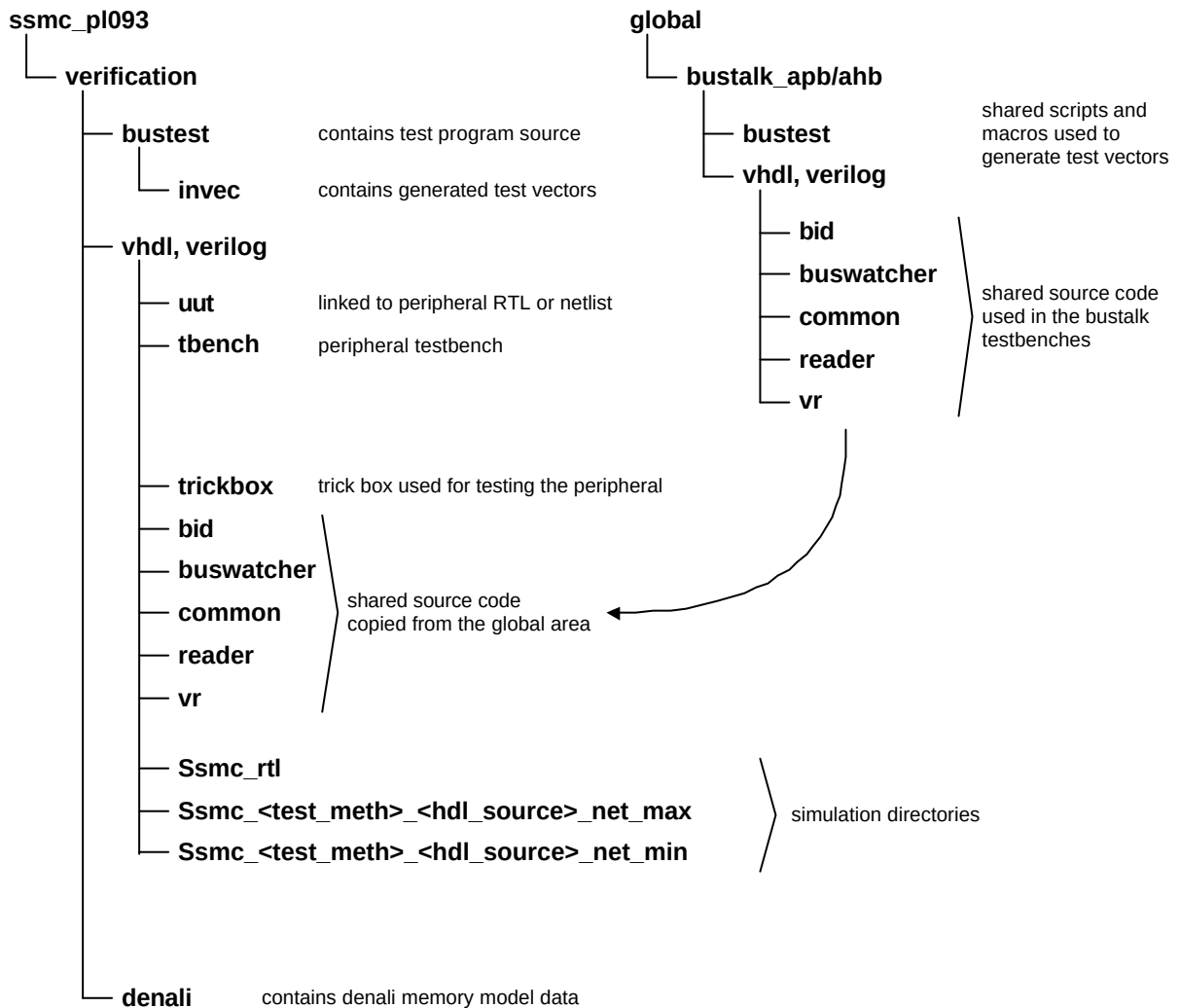


Figure 3-4: BusTalk verification directory structure

The **uut** directory is linked either to the RTL source or the netlist directory depending on the simulation being performed.

NOTE: Denali memory model SOMA files for the memory models used in the testbenches are provided in the `ssmc_pl093/verification/denali` directory. A Denali MMAV license is required to run the verification testbenches that use these Denali models. See the section entitled [Denali simulation models](#) on page 15 for more information on using the Denali memory models.

3.4 Integration test directory structure

The integration directory contains the integration test environment consisting of the testbench and the integration test program source. The testbench is a minimal EASY system supporting TIC testing.

Figure 3-5 illustrates the integration test directory structure:

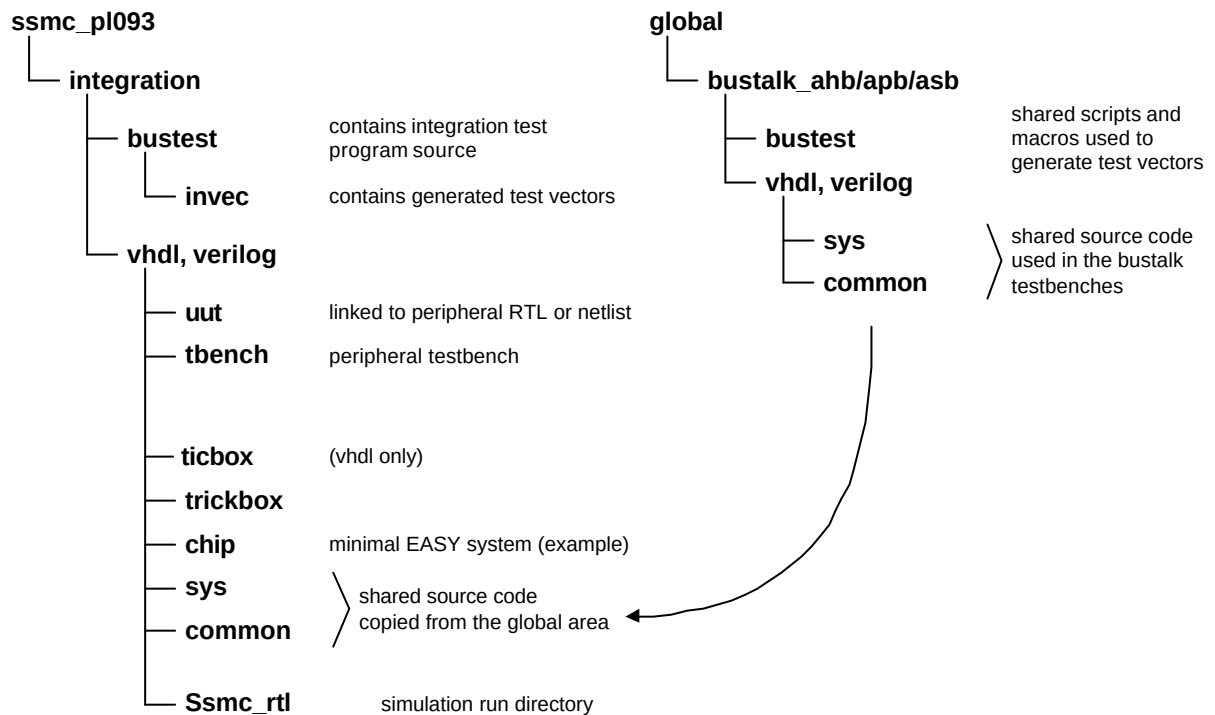


Figure 3-5: Integration testbench directory structure

4 USAGE INSTRUCTIONS

This section guides the user through the peripheral design environment and the use of the various design tools that are supported. This section is divided into the following sub-sections:

- Design tools used
- Reference cell library
- Peripheral development environment
- Running simulations and synthesis
- Error and warning messages

4.1 Design tools used

The *ARM PrimeCell Synchronous Static Memory Controller PL093 Release Note* defines the simulation and synthesis tools used in the development and testing of this product. Amendments to the configuration, scripts and make files will be required if different design tools are used.

Simulations have been performed with Model Technology Inc, ModelSim and Cadence Verilog-XL. The ModelSim simulator is an event-driven simulator which produces compiled environments for the HDL source. The Verilog-XL simulator is an event-driven simulator which interprets the HDL source at run-time.

Support for these simulators is provided by makefiles in the `global/modelsim` and `global/LDV` directories. Local modified versions of the global makefiles can also be found in the `verification/verilog/tbench` and `verification/vhdl/tbench` directories. If another simulator tool is used then another directory should be created i.e. `global/<simulator>` and a corresponding set of makefiles created by copying and editing one of the supplied set of makefiles. Similarly, new local makefiles in `verification/verilog/tbench` and `verification/vhdl/tbench` directories need creating to support another simulator.

For further information, refer to the README files within the `tbench` directories for each testbench.

4.2 Reference Cell library

The Avant! CB18 (0.18 μm) standard cell library is supplied with the peripheral as a reference cell library. This proprietary Avant! Passport standard cell library has been used during development of the peripheral, for synthesis and gate-level simulation with post-synthesis back-annotated timings in Standard Delay Format (SDF). For further information on this cell library, refer to the documentation supplied.

Use of a different technology library or design tools requires that script files and configuration files must be amended for the user's chosen technology library, simulation and synthesis tools. For further information, particularly regarding amendment of synthesis script files to a different technology library, and integration of peripherals in an AMBA-based ASIC, refer to the *PL093 SSMC Integration Manual*.

This reference cell library is provided to enable the user to replicate the synthesis and verification processes performed by ARM. Please note that this is a reference library only with no physical views, and users must provide their own target library for the development of their system.

The gate count and performance achieved relate to the reference cell library only and it is believed that this library is representative of a typical vendor cell library. Synthesis to the customers target library should yield similar gate counts, but performance is vendor library specific.

[Figure 4-1](#) illustrates the cell library directory structure. The top-level CB18 directory can be placed in the same directory level as the top-level of the peripherals, in order to run the gate-level simulations as defined in configuration files, shell scripts and usage instructions described later.

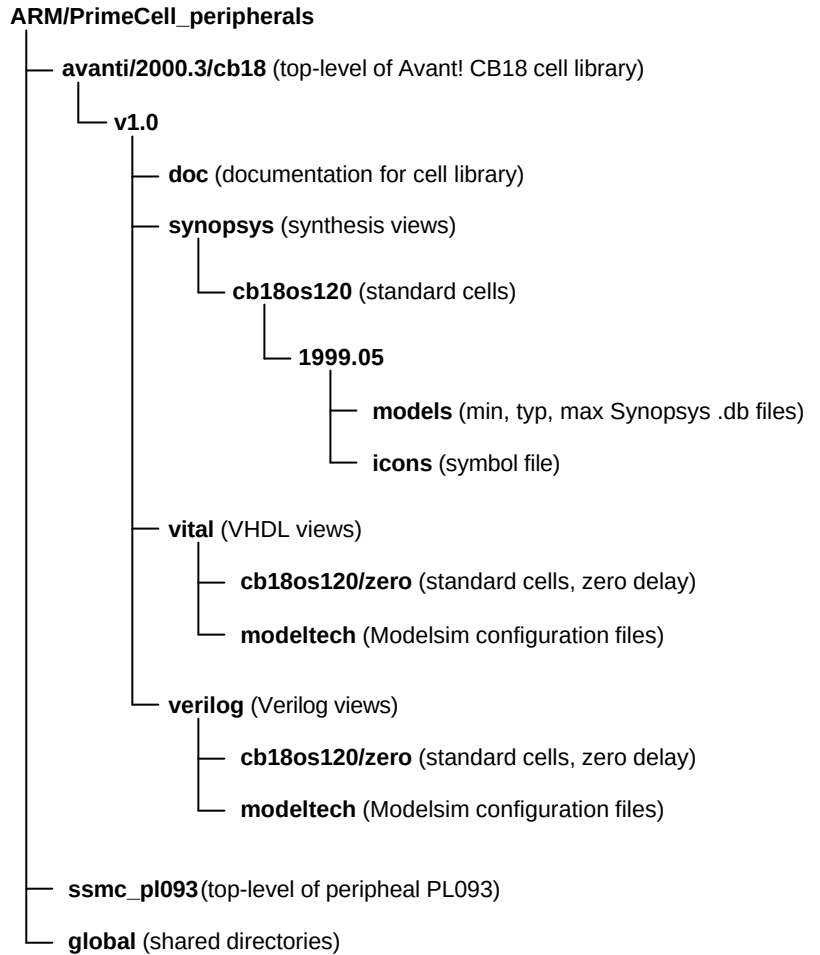


Figure 4-1: Avant! CB18 cell library directory structure

4.3 Peripheral Development Environment

The peripheral development environment is designed to support a number of standard tools within a defined directory structure. To control how these tools are used a number of Unix environment variables are defined and a system of makefiles are provided. This minimises the amount of user interaction required to support the tool chain.

To support the different requirements of the end user, a number of design parameters are supported. These are used to define which of the supported design flows is to be used. Additionally, environmental parameters are supported to allow the end user to customise the peripheral development environment, allowing integration with their own development environment.

In general, running a design tool (simulation, synthesis) consists of two steps:

- setting up environment variables
- using the Unix make tool to start a design tool

The make tool will then create necessary directory structure, setup files required and start the design tool in command line mode.

4.3.1 Unix environment variables

Unix environment variables are used to simplify the configuration of the directory paths, design tools and design flow options. A brief overview of the variables is given in [Table 4-1](#).

Table 4-1: Peripheral development Unix environment variables

Variable	Value	Description
GLOBAL	<global_dir_path>	Path to the shared global files
PERIPH	Ssmc	Peripheral name
SIMULATOR	modelsim, LDV	Simulator used
TEST_METH	noscan, scaninsert	Scan methodology to use for synthesis
HDL_SOURCE	Verilog, vhdl	RTL source code type
HDL_NETL	Verilog	Netlist type
TEST_ENV	BUSTALK, TICTALK	Test environment type
AMBA	AHB, APB	AMBA bus type. Only AHB is used for PL093.
LIB_VHDL	<vhdl_library_path>	Location of the VHDL cell library
DIRS_VERILOG	<verilog_library_path>	Location of the Verilog cell library
FILES_VERILOG	<verilog_udp_path>	Location of the Verilog UDP file
LIB_SYNOP	<synopsys_library_path>	Location of the Synopsys target library
TBENCH	<no value>, _denali, _denali1, _denali2	Testbench to use when running a BusTalk verification test. A specific testbench must be used with a specific BusTalk test

The peripheral's top-level makefile will overwrite the PERIPH variable with the correct name, but before running the make routine this variable must be declared within the Unix environment. This enables integration of the PrimeCell peripheral makefile structure into a project (ASIC) makefile structure, where multiple PrimeCell peripherals might exist and each one of them requires the PERIPH variable to have a different value.

4.3.2 Makefile commands

The makefile structure consists of the peripheral specific makefiles and the shared makefiles (located in the global directory, which is referenced by the GLOBAL environment variable). The peripheral makefiles are used to move through the peripheral directory structure, setup the environment and then execute a shared makefile. The peripheral makefiles consist of the top-level makefile located in the ssmc_pl093 directory and the makefiles in the peripheral sub-directories.

The shared makefiles are located in the \$GLOBAL directory, these makefiles are common to all peripherals in a system/ASIC and contain design tools specific commands. This enables an easy and quick modification to the makefile structure, if a design flow or a design tool needs to be changed.

Only the top-level makefile will be explained here, see the "README" and "makefile" files in the peripheral sub-directories for more details. A summary of the top-level makefile commands is given in [Table 4-2](#).

Table 4-2: Makefile arguments

Make Argument	Commands	Description
all	cd \$HDL_SOURCE make all	Compiles the RTL source code for simulation (for ModelSim only)
rtl	cd verification make rtl	Invokes the BusTalk testbench RTL simulation
netlist	cd synopsys make netlist	Invokes synthesis to produce a gate-level netlist
net_max	cd verification make net_max	Invokes the BusTalk testbench netlist simulation with maximum delays
net_typ	cd verification make net_typ	Invokes the BusTalk testbench netlist simulation with typical delays
net_min	cd verification make net_min	Invokes the BusTalk testbench netlist simulation with minimum delays
clean	cd verilog; make clean cd verification; make clean cd integration; make clean cd synopsys; make clean	Removes work files in the sub-directories

4.4 Denali simulation models

Denali memory models are used in the verification testbenches to provide accurate simulation models of real memory devices.

Denali memory models are third party memory models developed by DenaliSoft Inc., a company specialising in memory simulation models. If you are not already a Denali licensee then to be able to run the full set of verification tests, you need to apply for a 30-day evaluation licence from the Denali website (<http://www.denalisoft.com>). The Denali models SOMA files are supplied with the PL093 deliverables and reside in the `ssmc_pl093/verification/denali` directory.

Refer to the *ARM Synchronous Static Memory Controller PL093 Design Manual* for details of the testbenches and the Denali models used in each testbench.

4.4.1 Verilog-XL and Denali dynamically linked shared object libpli.so

When using the Cadence Verilog-XL simulator with Denali models, a Denali dynamically linked shared object file (`libpli.so`) is required to be loaded by the simulator at run time.

This `libpli.so` is not supplied with the PL093 deliverables but must be generated by the user for the specific version of Denali and Verilog-XL being used. This must be done before any BusTalk verification testbench tests can be run. Please refer to documentation in your Denali installation for instructions on generating this shared object file (see `$DENALI/verilog/README.dynamic`).

The `LD_LIBRARYPATH` environment variable must be set to include the path to this `libpli.so` file in order for the Verilog-XL simulator to load the shared object at run time. The path is added to the variable by the following command:

```
setenv LD_LIBRARYPATH <path_to_libpli>:$LD_LIBRARYPATH
```

4.5 Running simulations and synthesis

This chapter describes the step-by-step procedures for performing the various simulation and synthesis operations supported in the peripheral design flow. Please refer to [Table 4-1](#) for a summary of Unix environment variables used by makefiles and scripts.

The simulation and synthesis operations supported by the peripheral design flow are described in the following sections:

- Generating BusTalk functional test vectors
- Compiling peripheral RTL code
- Running BusTalk verification RTL simulations
- Running synthesis
- Running netlist simulations
- Generating the integration test vectors
- Running the integration test RTL simulation

4.5.1 Generating BusTalk functional test vectors

The BusTalk test programs are written in 'C' and located in the `verification/bustest` directory. The generated BusTalk (.sim and .bif) test vector files compiled from the C source are placed in the `verification/bustest/invec` directory. To generate the test vectors, first setup the following environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Ssmc
setenv SIMULATOR  modelsim, LDV
```

To compile a BusTalk test, change to the `ssmc_pl093` directory and type:

```
cd verification/bustest
make <test_name>
```

where <test_name> is one of the tests defined in `verification/bustest/makefile`

For example, the following command will compile the BusTalk vectors for carrying out register access tests on the SSMC :

```
make RegisterTests
```

A `bif.sim` test vector file will be created in the `verification/bustest/invec` directory which will be executed by the testbench at simulation time.

The BusTalk tests are divided into a number of separate modules which focus on a particular type of functional test. There are several testbenches, each with a different memory configuration and different timing settings are used for some tests.

There are several major groups of tests defined by the `verification/bustest/makefile` which make it convenient to run groups of tests as well as still giving the flexibility to compile individual specific tests. The major groups of tests can be compiled by invoking the following make commands from within the `verification/bustest` directory:

- `make all` (compiles all tests that do not require Denali memory models)
- three variants of tests that use Denali memory models:
 - `make DenaliTests`
 - `make DenaliTests1`
 - `make DenaliTests3`

The *ARM PrimeCell Synchronous Static Memory Controller PL093 Design Manual* details the purpose of each test and each testbench configuration.

4.5.2 Compiling peripheral RTL code

Ensure that the relevant tools have been sourced. To compile the peripheral RTL code for the ModelSim simulator, first set the environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Ssmc
setenv SIMULATOR  modelsim, LDV
setenv HDL_SOURCE  verilog, vhdl
setenv TEST_ENV    BUSTALK
```

To compile the RTL, change to the `ssmc_pl093` directory and type:

```
make all
```

This will compile the peripheral RTL code in the correct order.

NOTE: When using the Cadence Verilog-XL simulator the RTL source code is compiled at runtime so this `make all` step is not necessary.

4.5.3 Running BusTalk verification RTL simulations

Ensure that the relevant tools have been sourced and the Denali shared object file `libpli.so` is setup if Verilog-XL is being used, as described in [Denali simulation models](#) on page 15. Before running the functional simulation, ensure that the appropriate functional test vectors are generated as described in [Generating BusTalk functional test vectors](#) on page 17. Set the following environment variables before invoking any simulations:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Ssmc
setenv SIMULATOR  modelsim, LDV
setenv HDL_SOURCE  verilog, vhdL
setenv TEST_ENV    BUSTALK
setenv TBENCH      not set, _denali, _denali1, _denali2
```

The TBENCH variable is used to specify one of four different testbench configurations. Specific BusTalk tests must be run on specific testbenches and the `timing.v/vhd` file in the `verification/$HDL_SOURCE/tbench` directory must be copied manually from a specific timing file for the testbench. [Table 4-3](#) defines testbench files and TBENCH settings required to run the various tests:

Table 4-3: TBENCH settings and testbench file setup for tests

BusTalk test compiled	TBENCH setting	Testbench used	timing.v file
make all	unsetenv TBENCH	tbench.v	timing_tbench.v
make DenaliTests	setenv TBENCH _denali	tbench_denali.v	timing_denali.v
make DenaliTests1	setenv TBENCH _denali1	tbench_denali1.v	timing_denali.v
make DenaliTests3	setenv TBENCH _denali2	tbench_denali2.v	timing_denali.v

For example, to run the BusTalk Verilog simulation for the default "all" tests the procedure is:

- compile the BusTalk test vectors according to section 4.5.1 i.e. "make all" in the `verification/bustest` directory
- compile the RTL according to section 4.5.2 i.e. "make all" in the `ssmc_pl093` directory
- "unsetenv TBENCH" to select the testbench without Denali memory models
- "cp timing_tbench.v timing.v" in the `verification/$HDL_SOURCE/tbench` directory
- "make rtl" in the `verification` directory to run the simulation

As an example of running one of the Denali based tests, to run the BusTalk Verilog simulation for the "DenaliTests1" tests the procedure is:

- compile the BusTalk test vectors according to section 4.5.1 i.e. "make DenaliTests1" in the `verification/bustest` directory
- compile the RTL according to section 4.5.2 i.e. "make all" in the `ssmc_pl093` directory
- "setenv TBENCH denali1" to select the use of the `tbench_denali1.v` testbench
- "cp timing_denali.v timing.v" in the `verification/$HDL_SOURCE/tbench` directory
- "make rtl" in the `verification` directory to run the simulation

The VHDL simulation will terminate with the following message to signal the completion of the test:

```
** Failure: End of test
```

This is normal and is a consequence of the method used by the testbench to terminate the simulation.

There should be no errors reported during the simulation for a successful run, except during the initialisation period and the expected errors and warnings detailed in the section titled [Error and warning messages](#) on page 26. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations.

The simulation log file will be written to the `verification/${HDL_SOURCE}/${PERIPH}_rtl` directory. The filename of the simulation log created will be 'transcript' for ModelSim and 'verilog.log' for Cadence Verilog-XL. Simulation log files are included in the release for all the major groups of tests on both ModelSim and Verilog-XL simulators.

4.5.4 Running Synthesis

Ensure that the relevant tools have been setup.

Synopsys Design Compiler is used for the peripheral synthesis, which targets the Avant! CB18 standard cell library.

The `ssmc_pl093/synopsys/scripts` directory contains the peripheral specific synthesis scripts and the shared scripts are located in the `$GLOBAL/synopsys_shared/scr_common` directory.

To run synthesis, first set the environment variables:

```
setenv GLOBAL          <global_dir_path>
setenv PERIPH          Ssmc
setenv TEST_METH        noscan, scaninsert
setenv HDL_SOURCE       verilog, vhdL
setenv HDL_NETL         verilog
setenv LIB_SYNOP        <synopsys_lib_path>
```

Check that the `$GLOBAL/synopsys_shared/synopsys_cache` directory exists and is write enabled. The scripts do not automatically generate this directory so it must be created manually for the first time.

The `TEST_METH` variable selects the test methodology used for the synthesis:

<code>noscan:</code>	the peripheral will be synthesised without using scan enabled flip-flops
<code>scaninsert:</code>	the peripheral will be synthesised with scan enabled flip-flops and a scan chain inserted

The `HDL_SOURCE` variable selects the HDL source being compiled and the `HDL_NETL` variable controls the output format of the netlist.

NOTE: The scripts supplied only write out Verilog netlists even when the `$HDL_SOURCE` is VHDL. Therefore, simulations with a VHDL netlist are not directly supported in this release.

To run synthesis, change to the `ssmc_pl093` directory and type:

```
make netlist
```

Synthesis log and report files will be written in the `synopsys/log` and `synopsys/report` directories, respectively. See sections entitled [log directory](#) and [report directory](#) for more details of these directories and their contents.

Synthesis log and report files are included in the release for the combinations of `$TEST_METH` set to `noscan` and `scaninsert` and `$HDL_SOURCE` set to `verilog` and `vhdL`.

4.5.4.1 scr_common directory

The `scr_common` directory contains the scripts that are shared by the PrimeCell peripherals. The files listed below are referenced in the peripherals synthesis script and must always be present.

- `setup.scr` – this file is used to define various Synopsys variables that are to be used during synthesis e.g. bus naming style.
- `global.scr` – this file defines global synthesis parameters which must be applied to all the PrimeCell peripherals e.g. `set_max_area`.
- `amba_params.scr` – these files define the timing parameter to be used for the various types of AMBA interfaces.
- `ahb_slave.scr` – these files define the timing constraints to be used for the various types of AMBA interfaces.
- `library_<libraryname>.scr` – this file contains cell library related information and the operating conditions. Using a single file to define all the library information makes it simpler to target a different cell library.
- `name_rules.scr` – the Synopsys naming rules, used to change the names of the cells, nets and ports in the design.

4.5.4.2 scripts directory

The `scripts` directory is specific to the peripheral and contains the synthesis command script and the synthesis constraints file that apply only to the peripheral. This directory contains the following files ;

- `Ssmc.cmd` - this is the synthesis command script used for the peripheral. This command script references the scripts contained in this and `scr_common` directories.
- `Ssmc_scan.cmd` – this is the command script used specifically for scan insertion. If scan insertion is not required this command script will not be used.
- `Ssmc.scr` – this file defines the timing requirements for the peripheral's non-AMBA signals.
- `Ssmc_exceptions.scr` – this file contains all the timing exceptions e.g. false and multi-cycle paths.

4.5.4.3 log directory

The `log` directory is used to store the information generated about the synthesis run. Two files are created:

- `$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL)_synth.log` – this file is generated within the synthesis command script and is used to record the output from the checks performed on the design being synthesised i.e. “`check_timing`” and “`check_test`” both pre and post scan insertion.
- `$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL)_run.log` – the synthesis run log is generated by piping the output of the “`dc_shell`” into this file.

All errors in these files must be fixed and the number of warning must be reduced to a minimum. The synthesis run log file might have following errors, which should be ignored:

```
Error: You don't have a 'HDL-Compiler' license to remove.  
Error: You don't have a 'Test-Compiler' license to remove.
```

NOTE When the “`noscan`” option is defined, a Test Compiler license is not required since the ‘`check_test`’ command is not executed by the scripts.

4.5.4.4 report directory

The report directory contains the Synopsys reports generated from the synthesis run. The file names of the generated reports vary depending on the parameters defined for the synthesis run i.e. the source code type and the test requirement. This allows different configurations to be synthesised without over-writing existing report files.

The various report filename formats and contents are as follows:

- **Violation:** `$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).vio`
The violation report contains a summary of all the constraint violations, the report is generated by the “report_constraint -verbose -all_violators” command. The only acceptable violation is a design area violation.
- **Timing:** `$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).max/min`
The two timing reports are generated to enable the actual timing of the synthesised netlist to be examined. The max path report file (*.max) contains a timing report for the ten longest under worst case conditions and is generated using the “report_timing -delay max -path full -nworst 10” command. The min path report file (*.min) contains a timing report for the ten shortest paths under best case conditions and is generated using the “report_timing -delay min -path full -nworst 10” command.
- **Area:** `$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).area`
The area report contains the area for each of the modules in the design.
- **Latch:** `$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).latch`
The latch report contains information of all latches and timing loops found in the design, the report is generated using the “all_registers -level_sensitive” and “report_timing -loops” commands
- **Constraint:** `$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).ports/constr`
For further information the constraints used during synthesis are reported in the following two files. The ports report contains (*.ports) information about the constraints placed on each of the peripheral's top level ports and is generated using the “report_port -verbose” command. The constraint report (*.constr) contains general information about the netlist that has been created, this report is generated using the “report_design”, “report_constraint”, “report_clock” and “report_attribute -design” commands.
- **Scan chain:** `$(PERIPH)_scaninsert_$(HDL_SOURCE)_$(HDL_NETL).scan`
The scan chain report is created by the “Ssmc_scan.cmd” script and is only generated when scan insertion has been performed. The report details the expected fault coverage, the scan configuration and the scan chain order using the “report_test -coverage”, “report_test -configuration” and “report_test -scan_path” commands respectively.

4.5.5 Running netlist simulations

Ensure that the relevant tools have been setup and the Denali shared object file `libpli.so` is setup if Verilog-XL is being used, as described in [Denali simulation models](#) on page 15. Compile the required BusTalk test in the `bustests` directory as described in [Generating BusTalk functional test vectors](#) on page 17. When using the ModelSim simulator the verilog cell library must be compiled in the `avanti/2000.3/cb18/v1.0/verilog/cb18os120/zero/` directory.

Compile the cell library into a ModelSim "work" directory using the following commands:

```
vlib work
vlog -work work *.v
```

To run the netlist simulation, set the environment variables to their required values:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Ssmc
setenv SIMULATOR  modelsim, LDV
setenv TEST_METH   noscan, scaninsert
setenv HDL_SOURCE  verilog, vhdl
setenv HDL_NETL    verilog
setenv DIRS_VERILOG <verilog_cell_lib_path>
setenv LIB_VHDL    <vhdl_cell_lib_path>
```

Note that the `TEST_METH`, `HDL_SOURCE` and `HDL_NETL` variables should have the same values for the peripheral synthesis and the netlist simulation.

To run a netlist simulation, change to the `ssmc_pl093` directory and type:

```
make net_max      (simulation with maximum SDF timings)
make net_typ      (simulation with typical SDF timings)
make net_min      (simulation with minimum SDF timings)
```

A VHDL simulation will terminate correctly with the following message to signal the completion of the test:

```
** Failure: End of test
```

This is normal and is a consequence of the method used by the testbench to terminate the simulation.

There should be no errors reported during the simulation for a successful run, except during the initialisation period and the expected errors and warnings detailed in the section titled [Error and warning messages](#) on page 26. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations. The simulation log file will be placed under the `verification/$HDL_NETL` directory in one of the following directories:

```
<periph>_<test_meth>_<hdl_source>_net_max/ (maximum timing simulation log)
<periph>_<test_meth>_<hdl_source>_net_typ/ (typical timing simulation log)
<periph>_<test_meth>_<hdl_source>_net_min/ (minimum timing simulation log)
```

The simulation log file generated will be 'transcript' for ModelSim it is and 'verilog.log' for Cadence Verilog-XL. Netlist simulation logs files are included in the release for only a subset of tests, simulator and timing variations (`net_max` and `net_min` only).

4.5.6 Generating the integration test vectors

To generate the integration test vectors, first set the environment variables as follows:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Ssmc
setenv SIMULATOR  modelsim, LDV
```

To compile the integration test vectors, change to the `ssmc_pl093` directory and type:

```
cd integration
make default
```

This will create the integration test vectors in the `ssmc_pl093/integration/bustest/invec` directory.

NOTE:	The integration tests supplied are only an example and should be modified when the peripheral is integrated into a system. Please see the peripheral's integration manual for more details.
--------------	---

4.5.7 Running the integration test RTL simulation

Ensure that the relevant tools are setup.

To perform the integration test simulation, first set the environment variables as follows:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Ssmc
setenv SIMULATOR  modelsim, LDV
setenv HDL_SOURCE   verilog, vhdl
unsetenv TBENCH
```

To run the integration test simulation, change to the `ssmc_pl093` directory and type:

```
cd integration
make rtl
```

A VHDL simulation will terminate with the following message to signal the successful completion of the test:

```
** Failure: End of test
```

This is normal and is a consequence of the method used by the testbench to terminate the simulation.

The simulation log file will be written to the `ssmc_pl093/integration/$HDL_SOURCE/Ssmc_rtl` directory.

4.6 Error and warning messages

This section lists the error and warning messages that exist in the simulation logs provided. These are expected and do not affect the validity of the tests.

4.6.1 BusTalk RTL verification simulations

The following error and warning messages are expected in the log files in the `ssmc_pl093/verification/verilog/Ssmc_rtl` directory:

Verilog/VHDL errors/warnings:

```
# BWAREBT: Master may have cancelled the burst before finishing all the
transfers. HADDR : 10100004. TIME :      17985 ns

# *Denali* Error: Memory Instance: tbench_denali2.uflash3 Time: 24590 ns
# Setup Violation (tavkh=9 ns) of 4 ns on signal a
```

The following error message can be ignored as the Read burst was terminated by a new burst.

```
# ** Warning: SSMCTM34: Access time violation for Read

# ** Warning: SSMCTM35: Maximum time limit for the burst read access has
exceeded
```

VHDL specific errors/warnings:

This occurs when 64 bit access is done on Memory port to cover the error response.

```
# ** Error: BWERRHSDW : The size of attempted transfer is greater than
databuswidth. HADDR : 20000000. TIME : 37054085.52 ns      11135.52 ns

# ** Error: HWDATAU : HWDATA is unknown at TIME : 37054335.52 ns

# ** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand, the
result will be 'X'(es).
```

4.6.2 Bustalk netlist simulations

Verilog-XL SDF back-annotation errors/warnings:

Many instances of the following type of error and warning message are to be expected:

```
../uut/Ssmc_scaninsert_verilog.sdf21 L104: SDFA Error: Unable to find
source port tbench_denali1.uut.HSIZESMC[1]

../uut/Ssmc_scaninsert_verilog.sdf21 L152602: SDFA Warning: Negative
timing check limit -4.500000 set to 0
```

4.6.3 Integration test simulations

VHDL simulation:

```
# ** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand, the  
result will be 'X'(es)!
```

```
# ** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand!
```

< End of document >